

Introduction to MAPREDUCE

Introduction

Mapreduce is a programming model and an associated implementation of for processing and generating large data sets with a parallel, distributed algorithm on a cluster.

• A MapReduce program is composed of a Map() procedure (method) that performs filtering and sorting.

And a Reduce() method that performs a summary operation.

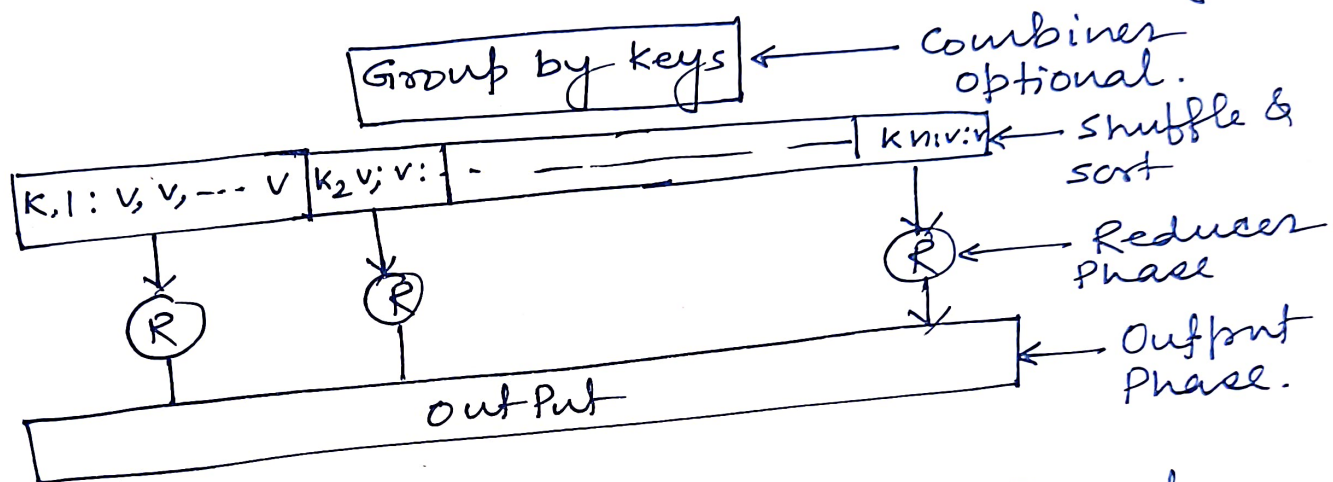
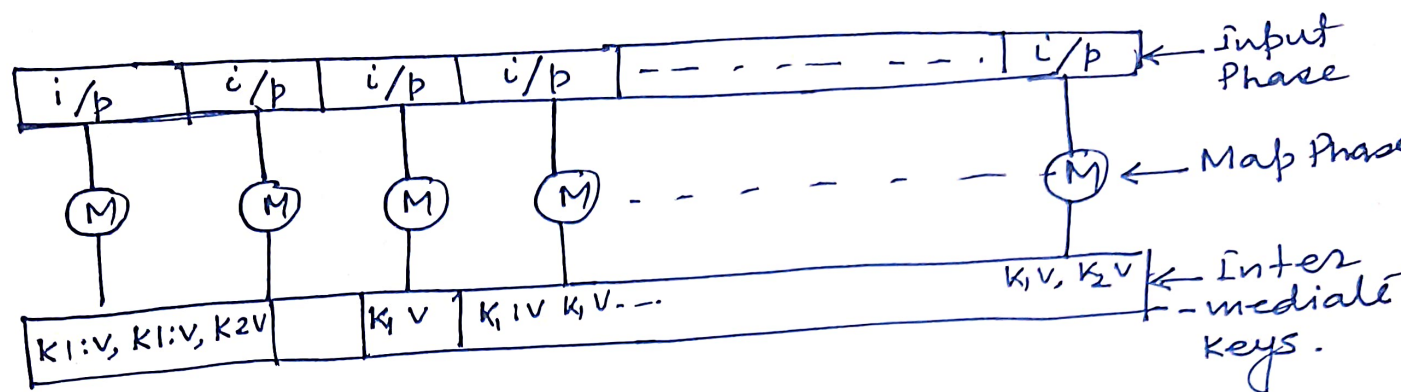
MapReduce is a framework for processing parallelizable problems across huge datasets using a large number of computers (nodes) collectively referred to as a cluster (if all nodes are on the same local network and use similar hardware) or a grid (if the nodes are shared across geographically and administratively distributed system, and use more heterogeneous hardware).

Processing can ~~be~~ occur on data stored either in a file system (unstructured) or in a database (structured)

The action of MapReduce framework can be divided into three steps:

- * **Map step:** Each worker node applies the `map()` function to the local data and writes the output to a temporary storage. A master node orchestrates that for redundant copies of input data, only one is processed.
- * **shuffle step:** Worker nodes redistribute data based on the output keys (produced by the `map()` function) such that all data belonging to one key is located on the same worker node.
- * **Reduce step:** Worker nodes now process each group of output data, per key, in parallel.

The entire process can be represented schematically as follows:



Input Phase: Here we have a Record Reader that translates each record in an input file and sends the parsed data to a mapper in the form of key-value pair.

Map - Map is a user-defined function, which takes a series of key-value pairs and processes each one of them to generate zero or more key-values pairs.

Intermediate Keys: The key-value pairs generated by the mapper are known as intermediate keys.

Combiner: A combiner is a type of local Reducer that group similar data from the map phase into identifiable sets. It takes the intermediate keys from the mapper as input and applies a user defined code to aggregate of the values in a small scope of one mapper. It is not a part of the main MapReduce algorithm, it is optional.

Shuffle and Sort

The Reducer task starts with the Shuffle and Sort step. It downloads the grouped key-value pairs onto the local machine, where the reducer is running. The individual key-value pairs are sorted by key into a larger data list. The data list groups the equivalent keys together so that their values can be iterated easily in the Reducer task.

Reducer

The Reducer takes the grouped key value paired data as input and runs a Reducer function on each one of them. Here the data can be aggregated, filtered, and combined in a number of ways, and it requires a wide range of processing. Once the execution is over, it gives zero or more key-value pairs to the final steps.

output phase

In the output phase, we have an output formatter that translates the final key-value pairs from the Reducer function and write them onto a file using a record writer.

Let us now discuss the MAP and Reduce operation in details:

~~Map operation in MapReduce.~~

Hadoop's MapReduce

MapReduce applications can process large data sets in parallel by using a large number of computers known as clusters.

In this programming paradigm, applications are divided into self-contained units of work. Each of these units of work can be run on any node in the cluster. In a Hadoop cluster, a MapReduce program is known as a job. A job is run by being broken down into pieces, known as tasks. These tasks are scheduled to run on the nodes in the cluster where the data exists.

Applications submit jobs to a specific node in a Hadoop cluster, which is running a program known as the Job Tracker. The Job Tracker programme communicates with the Name Node to determine where all of the data

required for the job exists across the cluster. The job is then broken into map tasks and reduce tasks for each node in the cluster to work on.

The Job Tracker program attempts to schedule tasks on the cluster where the data is stored, rather than sending data across the network to complete a task. The MapReduce framework and the Hadoop Distributed File System (HDFS) typically exist on the same set of nodes, which enables the Job Tracker program to schedule tasks on nodes where the data is stored.

Job Tracker

The Job Tracker is the service within Hadoop that farms out MapReduce tasks to specific nodes in the cluster, ideally the nodes that have the data, or at least are in the same rack.

1. Client applications submit jobs to the Job Tracker.
2. The Job Tracker talks to the NameNode to determine the location of the data.
3. The Job Tracker locates Task Tracker nodes with available slots at or near the data.
4. The Job Tracker submits the work to the chosen Task Tracker node.

5. The TaskTracker nodes are monitored. If they do not submit heartbeat signals often enough, they are ~~not~~ deemed to have failed and the work is scheduled on a different Task Tracker.
6. A Task Tracker will notify the Job Tracker when a task fails. The Job-Tracker decides what to do then: it may resubmit the job elsewhere, it may mark that specific record as something to avoid, and it may even blacklist the Task Tracker as unreliable.
7. When the work is completed the Job Tracker updates the status.
8. Client applications can poll the Job Tracker ~~updates the status~~ for information.

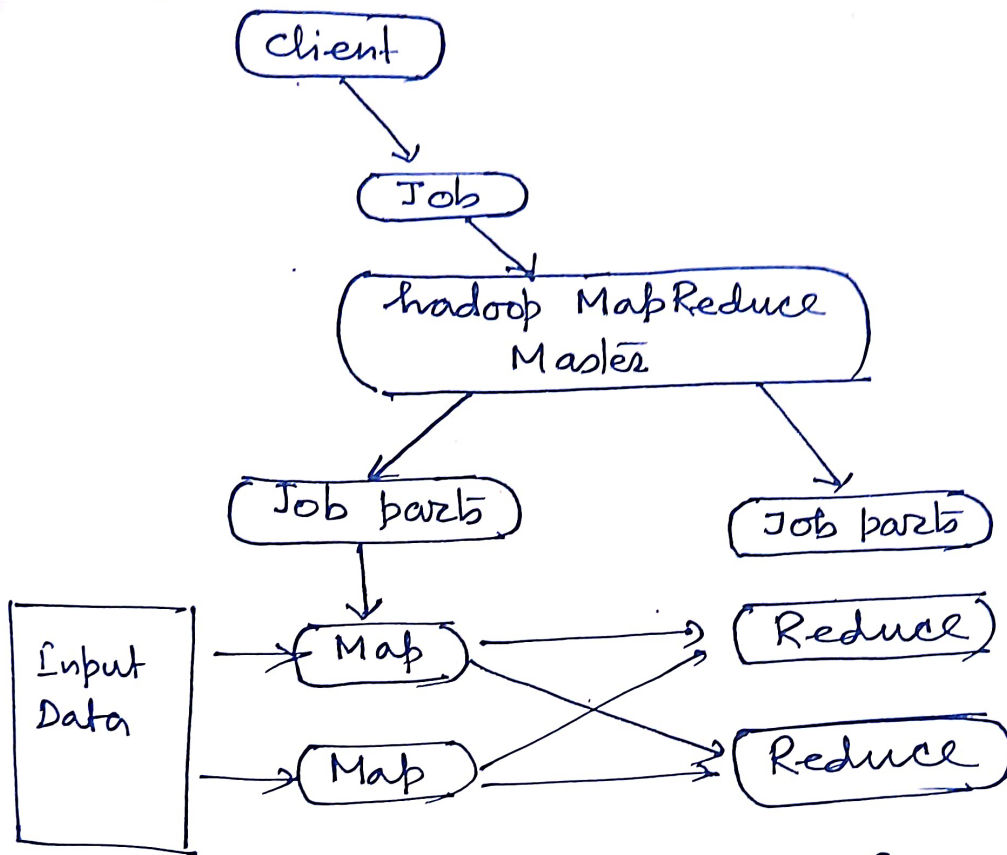
and Reduce

Q:-1 Describe the Map function operation in MapReduce (10)

Introduction

Ans: In Big Data we actually process huge data using a distributed data processing framework. MapReduce and Hadoop HDFS are the two major components of Hadoop which makes so powerful and efficient to use. MapReduce is a programming model used for efficient processing in parallel over large data-sets in a distributed manner. The architecture of the Map reduce programming framework

is given below:



The MapReduce programming framework is divided into two phases, namely the mapper ① Map phase ② Reduce Phase.

Map Phase:

The Map task takes a set of data and converts it into another set of data. The output of the mapper is data represented using a special data structure, known as key-value pair. It accepts the input data stored in using the same key-value data structure.

The key-value data structure is ~~sim~~ very similar to the ~~key~~ Array data structure. In array data are stored into contiguous memory locations indexed by number. So any data can be represented by (index, value). same concept is used in key-value data structure. For example in a student data set the Roll no of the student is the key ~~value~~ where as the name of the data is the value.

($\frac{\text{Roll}}{\text{key}}, \frac{\text{Name}}{\text{value}}$)

⇒ In the map phase the program converts the data into (key-value) format. This is done using Java collection framework.

The advantage of mapping the data into key-value pair is as follows: are the following

i) Scalability:

scalability refers to how quickly

i) scalability: It offers quicker and easy way to scan and process data.

ii) Speed: It offers easy way to read and write data.

iii) Flexibility: Key-Value pairs. ~~for~~ for faster speedy and scalable key-value stores, which allows.